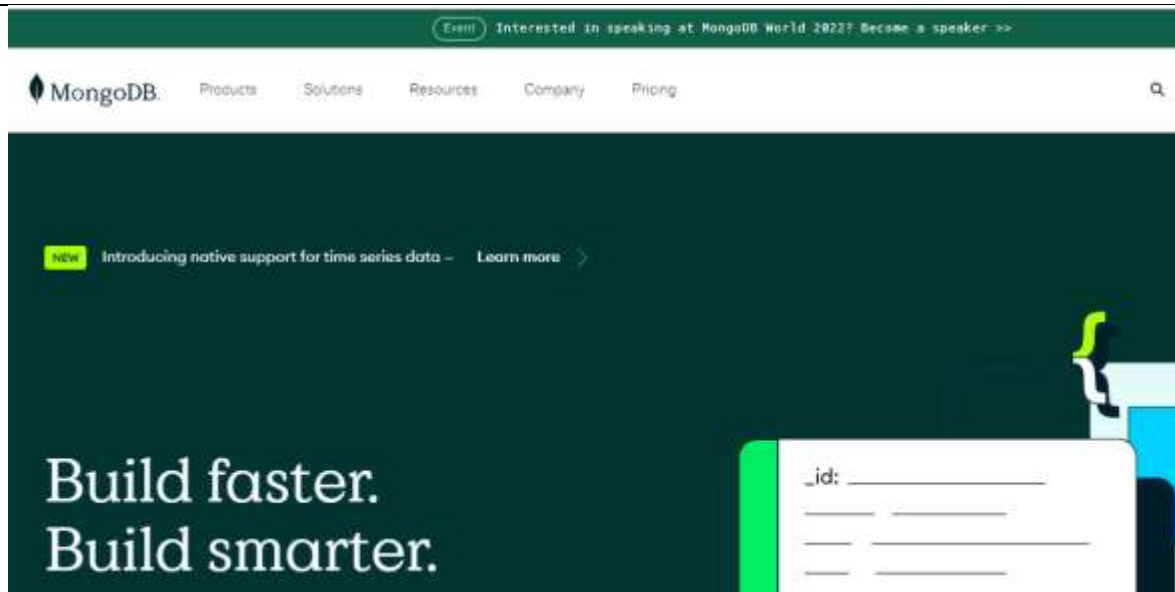


	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

		FORMATO DE GUÍA DE PRÁCTICA DE LABORATORIO / TALLERES / CENTROS DE SIMULACIÓN – PARA DOCENTES	
CARRERA: Ingeniería en Ciencias de la Computación		ASIGNATURA: Programación y Plataformas Web	
NRO. PRÁCTICA:	5	TÍTULO PRÁCTICA: Desarrollo de un CRUD usando MongoDB y Express.js	
OBJETIVO Desarrollar un CRUD con Express.js y MongoDB			
INSTRUCCIONES (Detallar las instrucciones que se dará al estudiante):		1. Conocer el funcionamiento de una base de datos MongoDB	
		2. Realizar la conexión a la base de datos desde un back-end	
		3. Implementar una arquitectura MVC	
ACTIVIDADES POR DESARROLLAR			
			
MongoDB (del inglés humongous, "enorme") es un sistema de base de datos NoSQL orientado a documentos de código abierto y escrito en C++, que en lugar de guardar los datos en tablas lo hace en estructuras de datos BSON (similar a JSON) con un esquema dinámico. Al ser un proyecto de código abierto, sus binarios están disponibles para los sistemas operativos Windows, GNU/Linux, OS X y Solaris y es usado en múltiples proyectos o implementaciones en empresas como MTV Network, Craigslist, BCI o Foursquare. La razón de esto es que MongoDB, al estar escrito en C++, cuenta con una más que notoria capacidad para aprovechar los recursos de la máquina y, al estar licenciado bajo una licencia GNU AGPL 3.0, es posible adaptarlo a nuestras necesidades. La información oficial sobre MongoDB se puede revisar en https://www.mongodb.com/			

	Computación	Docente: Ing. Patsy Malena Prieto MsC
	Programación y Plataformas Web	Período Lectivo: Septiembre 2022 – Febrero 2023



Conceptos Básicos en una base de datos MongoDB

Base de Datos una base de datos no relacional es un conjunto de datos llamados colecciones o collection.

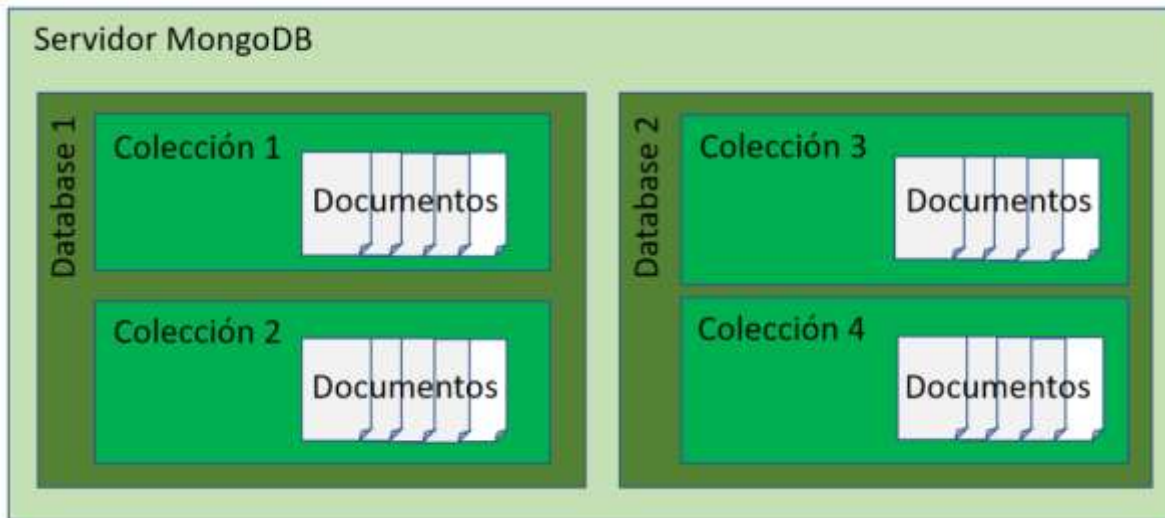
Collection Las colecciones representan la agrupación de datos similares que se realizará.

Db creation collections

Documents Los documentos son cada uno de los datos almacenados en una colección. Estos documentos serán archivos JSON

```
{
  "nombre": "laptop",
  "precio": 490.2,
  "activo": false,
  "creado": new Date ("12/12/1999"),
  "proveedor": {
    "nombre": "dell",
    "version": "xps",
    "ubicacion": {
      "ciudad": "Quito",
      "direccion": "cci local 23"
    }
  }
}
```

	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		



MODULO MONGOOSE

→ npmjs.com/package/mongoose

♥ Nameless Package Manager

npm Search packages

mongoose TS
6.1.6 • Public • Published a day ago

[Readme](#) [Explore](#) BETA [10 Dependencies](#)

Mongoose

Mongoose is a **MongoDB** object modeling tool designed to work in an asynchronous environment. Mongoose supports both promises and callbacks.

[Slack Status](#) [Test passing](#) [npm package 6.1.6](#)

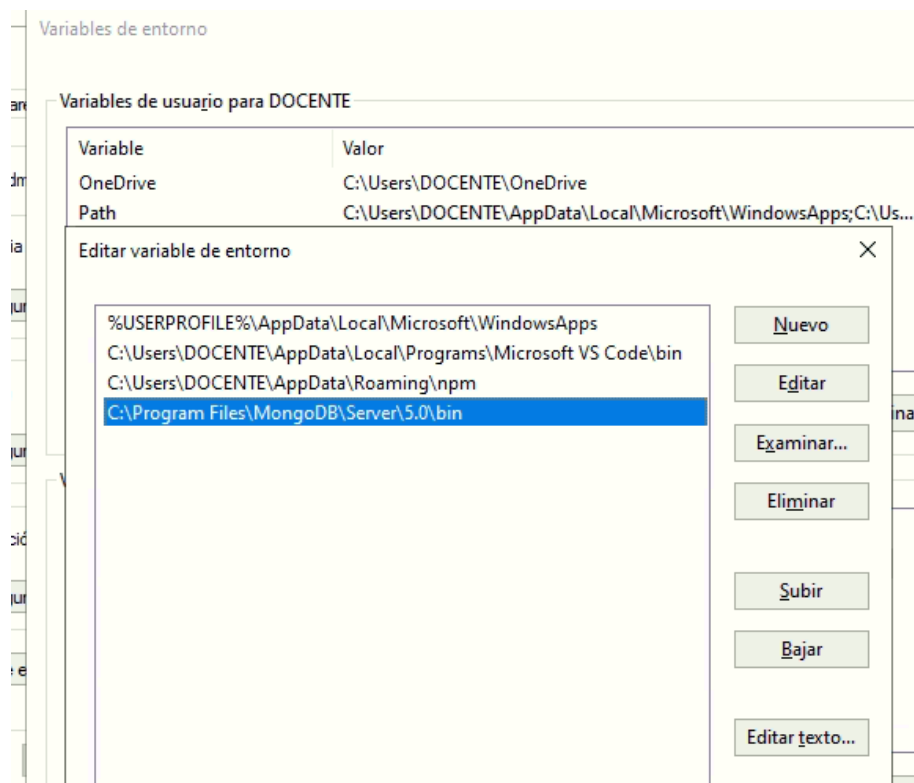
 npm install mongoose

Mongoose es una biblioteca de programación orientada a objetos de JavaScript que crea una conexión entre MongoDB y el marco de la aplicación web Express.

1. Luego de instalar MongoDB se puede verificar su correcto funcionamiento revisando la versión de la base de datos con el comando ***mongod -version*** en la ubicación donde se ha instalado la base de datos.

```
C:\Program Files\MongoDB\Server\8.0\bin>mongod --version
db version v8.0.10
Build Info: {
  "version": "8.0.10",
  "gitVersion": "9d03076bb2d5147d5b6fe381c7118b0b0478b682",
  "modules": [],
  "allocator": "tcmalloc-gperf",
  "environment": {
    "distmod": "windows",
    "distarch": "x86_64",
    "target_arch": "x86_64"
  }
}
```

En caso de algún error es necesario configurar la variable de ambiente Path con la ubicación de instalación de MongoDB. Generalmente se instala en *C:\Program Files\MongoDB\Server\5.0\bin*



2. Crear una carpeta en *C:/data/bd* en la cual se guardarán los datos de la base de datos

	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		



3. Ejecutar el comando *mongod*, que permite inicializar el servidor de MongoDB. Esperando conexiones en el puerto 27

```

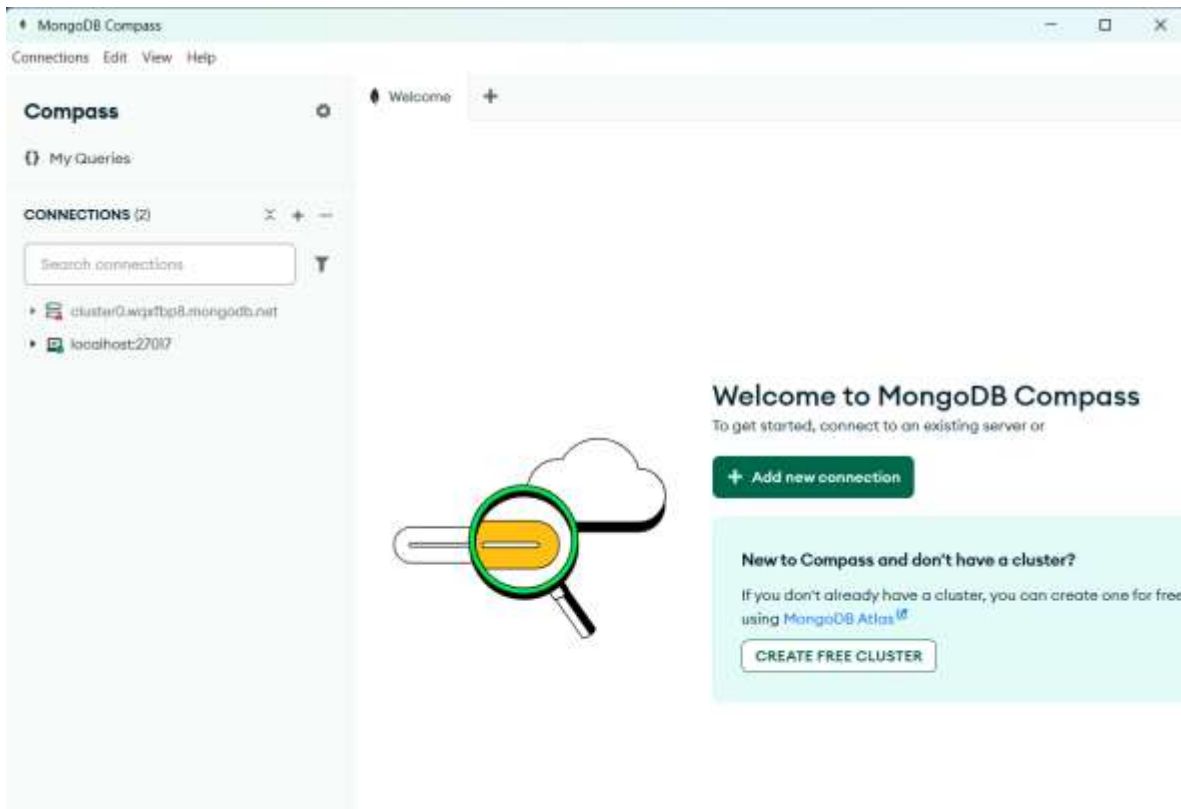
C:\> Símbolo del sistema - mongod
version":"5.0"}
{"t":{"$date":"2022-01-04T18:59:45.191-05:00"},"s":"I", "c":"NETWORK", "i
comingExternalClient":{"minWireVersion":0,"maxWireVersion":13},"incomingInt
reVersion":13},"isInternalClient":true},"newSpec":{"incomingExternalClient
xWireVersion":13},"outgoing":{"minWireVersion":13,"maxWireVersion":13},"isI
{"t":{"$date":"2022-01-04T18:59:45.196-05:00"},"s":"I", "c":"NETWORK", "i
comingExternalClient":{"minWireVersion":0,"maxWireVersion":13},"incomingInt
WireVersion":13},"isInternalClient":true},"newSpec":{"incomingExternalClien
maxWireVersion":13},"outgoing":{"minWireVersion":13,"maxWireVersion":13},"i
{"t":{"$date":"2022-01-04T18:59:45.198-05:00"},"s":"I", "c":"STORAGE", "i
{"t":{"$date":"2022-01-04T18:59:45.201-05:00"},"s":"I", "c":"CONTROL", "i
{"t":{"$date":"2022-01-04T18:59:47.406-05:00"},"s":"W", "c":"FTDC", "i
", "attr":{"error":{"code":179,"codeName":"WindowsPdhError","errmsg":"PdhAdd
{"t":{"$date":"2022-01-04T18:59:47.408-05:00"},"s":"I", "c":"FTDC", "i
ttr":{"dataDirectory":"C:/data/db/diagnostic.data"}}
{"t":{"$date":"2022-01-04T18:59:47.414-05:00"},"s":"I", "c":"STORAGE", "i
up_log","uuidDisposition":"generated","uuid":{"uuid":{"$uuid":"644ed772-7a8
{"t":{"$date":"2022-01-04T18:59:47.448-05:00"},"s":"I", "c":"INDEX", "i
ll,"namespace":"local.startup_log","index":"_id","commitTimestamp":null}}
{"t":{"$date":"2022-01-04T18:59:47.464-05:00"},"s":"I", "c":"NETWORK", "i
{"t":{"$date":"2022-01-04T18:59:47.464-05:00"},"s":"I", "c":"NETWORK", "i
f"}}
{"t":{"$date":"2022-01-04T18:59:47.472-05:00"},"s":"I", "c":"CONTROL", "i
ng until next sessions reap interval","attr":{"error":"NamespaceNotFound: c
{"t":{"$date":"2022-01-04T18:59:47.474-05:00"},"s":"I", "c":"STORAGE", "i
:"config.system.sessions","uuidDisposition":"generated","uuid":{"uuid":{"$u
{"t":{"$date":"2022-01-04T18:59:47.521-05:00"},"s":"I", "c":"INDEX", "i
buildUUID":null,"namespace":"config.system.sessions","index":"_id","commit
{"t":{"$date":"2022-01-04T18:59:47.521-05:00"},"s":"I", "c":"INDEX", "i
buildUUID":null,"namespace":"config.system.sessions","index":"_lsidTTLIndex"
{"t":{"$date":"2022-01-04T19:00:45.120-05:00"},"s":"I", "c":"STORAGE", "i
119388][14260:140721463973200], WT_SESSION.checkpoint: [WT_VERB_CHECKPOINT_
mestamp: (0, 0) , meta checkpoint timestamp: (0, 0) base write gen: 1"]}}
{"t":{"$date":"2022-01-04T19:01:45.144-05:00"},"s":"I", "c":"STORAGE", "i
144587][14260:140721463973200], WT_SESSION.checkpoint: [WT_VERB_CHECKPOINT_
mestamp: (0, 0) , meta checkpoint timestamp: (0, 0) base write gen: 1"]}}

```

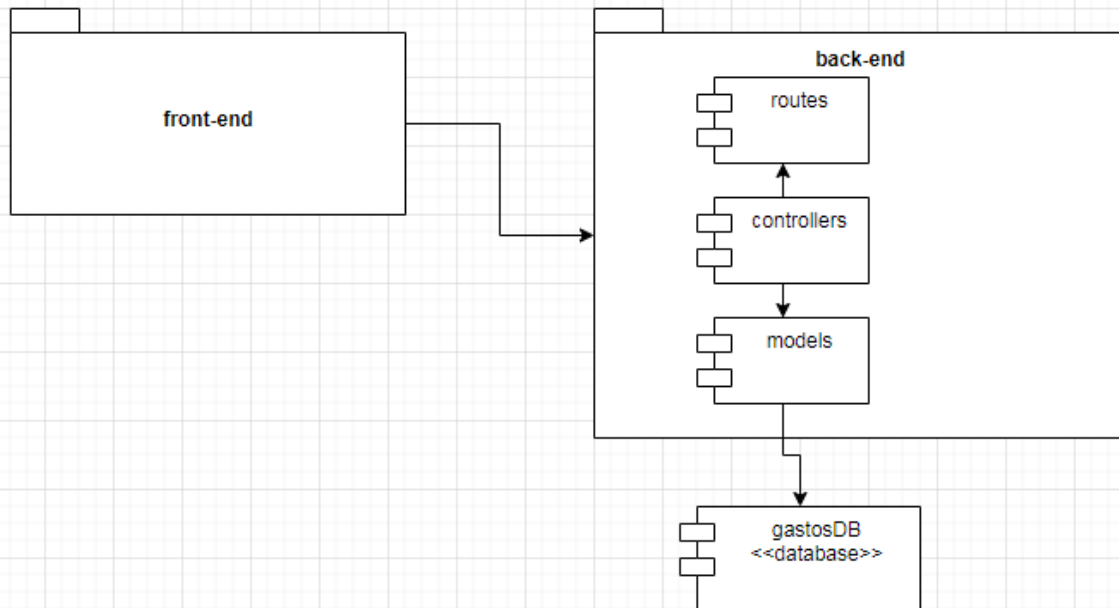
4. Para ingresar a la consola de comando en MongoDB se usa la sentencia *mongod*. O la interfaz gráfica MongoCompass

	Computación	Docente: Ing. Patsy Malena Prieto MsC
	Programación y Plataformas Web	Período Lectivo: Septiembre 2022 – Febrero 2023

```
C:\Program Files\MongoDB\Server\8.0\bin>mongod
{"t":{"$date":"2025-07-01T15:56:53.907-05:00"},"s":"I", "c":"CONTROL", "id":23285, "ctx":"thread1","msg":"Automatically disabling TLS 1.0, to force-enable TLS 1.0 specify --sslDisabledProtocols 'none'"}
{"t":{"$date":"2025-07-01T15:56:53.911-05:00"},"s":"I", "c":"CONTROL", "id":5945603, "ctx":"thread1","msg":"Multi threading initialized"}
{"t":{"$date":"2025-07-01T15:56:53.912-05:00"},"s":"I", "c":"NETWORK", "id":4648601, "ctx":"thread1","msg":"Implicit TCP FastOpen unavailable. If TCP FastOpen is required, set at least one of the related parameters","attr":{"relatedParameters":{"tcpFastOpenServer","tcpFastOpenClient","tcpFastOpenQueueSize"}}}
{"t":{"$date":"2025-07-01T15:56:53.913-05:00"},"s":"I", "c":"NETWORK", "id":4915701, "ctx":"thread1","msg":"Initialized wire specification","attr":{"spec":{"incomingExternalClient":{"minWireVersion":0,"maxWireVersion":25},"incomingInternalClient":{"minWireVersion":0,"maxWireVersion":25},"isInternalClient":true}}}
{"t":{"$date":"2025-07-01T15:56:53.915-05:00"},"s":"I", "c":"TENANT_M", "id":7091600, "ctx":"thread1","msg":"Starting TenantMigrationAccessBlockerRegistry"}
{"t":{"$date":"2025-07-01T15:56:53.916-05:00"},"s":"I", "c":"CONTROL", "id":4615611, "ctx":"initandlisten","msg":"MongoDB starting","attr":{"pid":22560,"port":27017,"dbPath":"/data/db","architecture":"64-bit","host":"UIOSLPT5ISPP"}}
{"t":{"$date":"2025-07-01T15:56:53.916-05:00"},"s":"I", "c":"CONTROL", "id":23398, "ctx":"initandlisten","msg":"Target operating system minimum version","attr":{"targetMinOS":"Windows 7/Windows Server 2008 R2"}}
{"t":{"$date":"2025-07-01T15:56:53.916-05:00"},"s":"I", "c":"CONTROL", "id":23403, "ctx":"initandlisten","msg":"Build info","attr":{"buildInfo":{"version":"8.0.10","gitVersion":"9d93076bb2d5147d5b66fe381c7118b0b8478b682","modules":[],"allocator":"tcmalloc-gperf","environment":{"distmod":"windows","distarch":"x86_64","target_arch":"x86_64"}}}}}
{"t":{"$date":"2025-07-01T15:56:53.916-05:00"},"s":"I", "c":"CONTROL", "id":51765, "ctx":"initandlisten","msg":"Operating System","attr":{"os":{"name":"Microsoft Windows 10","version":"10.0 (build 26100)}}}
{"t":{"$date":"2025-07-01T15:56:53.916-05:00"},"s":"I", "c":"CONTROL", "id":21951, "ctx":"initandlisten","msg":"Options set by command line","attr":{"options":{}}}
```



5. Crear un proyecto que contendrá la información del backend y frontend con la siguiente estructura



6. Para inicializar el proyecto e incluir las dependencias iniciales que corresponden al BACKEND se usarán los siguientes comandos

Comandos	Explicación de uso
npm init --yes	Nodejs
npm install express npm i express	Instalación de Express en el proyecto
npm install nodemon -D	Instalación de Nodemon como dependencia de desarrollo
npm run dev	Levantar la dependencia de Nodemon en modo desarrollo
npm install morgan	Instalación de Morgan
npm install mongoose	Instalación de Mongoose, módulo para conectarse a MongoDB y modelar los datos de la base de datos

En el archivo de configuración package.json se puede verificar que todos los módulos se han instalado correctamente. Además es necesario modificar en el apartado *scripts* con el código mostrado para que Nodemon se ejecute.

site002 > {} package.json > {} scripts > dev

```
1 {
2   "name": "site002",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "dev": "nodemon server/index.js"
8   },
9   "keywords": [],
10  "author": "",
11  "license": "ISC",
12  "dependencies": {
13    "express": "^4.17.2",
14    "mongoose": "^6.1.5",
15    "morgan": "^1.10.0"
16  },
17  "devDependencies": {
18    "nodemon": "^2.0.15"
19  }
20 }
```

7. Para organizar la información en el proyecto se creará las siguientes carpetas colocadas en la carpeta server. Para el manejo del servidor se creará el archivo *servidor.js* y para la base de datos se creará el archivo *database.js*



8. La página inicial que nos permite organizar la ejecución de la aplicación Express.js es el archivo *servidor.js*. En este archivo se colocará la siguiente estructura:


```
site002 > server > JS index.js > ...
1  const express=require('express');
2  const app=express();
3  const morgan=require('morgan');
4  const {mongoose}=require('./database');
5
6  //settings
7  app.set('nombreApp','Aplicacion para manejo de gastos SRI');
8  app.set('puerto',process.env.PORT|| 3000);
9
10
11 //middleware
12 app.use(morgan('dev'));
13 app.use(express.json());
14
15
16 //routes
17 app.use('/api/gastos',require('./routes/server.routes'));
18
19 app.listen(app.get('puerto'), ()=>{
20   console.log('Nombre de la App',app.get('nombreApp'));
21   console.log('Puerto del servidor',app.get('puerto'));
22 })
--
```

9. Para establecer la conexión con la base de datos MongoDB se usará el script **server/database.js**. Se debe colocar la cadena de conexión considerando la máquina local y el nombre de la base de datos.

```
const mongoose=require('mongoose');
const URI='mongodb://localhost/gastos';
mongoose.connect(URI)
.then(db=> console.log('BD conectada'))
.catch(err => console.error(err));
```

module.exports=mongoose;

10. En el script **servidor.js** incluir el llamado a este script

```
const {mongoose}=require('./database');
```

11. En el script **servidor.js** se colocará el código para usar el script con routes.

```
app.use(require('./routes/server.routes'));
```

12. En el archivo **routes/server.routes.js** se organizan las rutas

```
const express= require('express');
const router=express.Router();

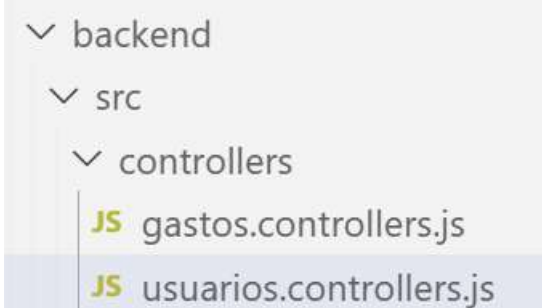
const gasto=require('../controllers/gastos.controllers');
const usuario=require('../controllers/usuarios.controllers');

router.get('/misitio/gastos',gasto.getGastos);
router.post('/misitio/gastos', gasto.addGasto);
```

```
router.get('/misitio/usuarios', usuario.getUsuarios);
router.post('/misitio/usuarios', usuario.addUsuario);
router.get('/misitio/usuarios/:id', gasto.getUsuario);
router.put('/misitio/usuarios/:id', gasto.editUsuario);
router.delete('/misitio/usuarios/:id', gasto.deleteUsuario);
```

```
module.exports=router;
```

13. En el directorio **controllers** se crean dos archivos **gastos.controllers.js** y **usuarios.controllers.js**



14. Usando los controllers se establecerá los endpoints que permitan ejecutar los distintos métodos HTTP. En el archivo **src/controllers/gastos.controller.js**. Se colocará el método **getGastos** que devuelve todos los gastos y el método **addGasto** que permita incluir un nuevo gasto en la base de datos.

```
const Gasto=require('../models/gastos');
const gastosControllers={};

//metodo GET
gastosController.getGastos= async(req, res)=>
{
  const gastos= await Gasto.find();
  res.json(gastos);
}

//metodo POST
gastosController.addGasto= async(req,res)=>{
  const gasto=new Gasto({
    id: req.body.id,
    tipo: req.body.tipo,
    monto:req.body.monto,
    descripcion:req.body.descripcion
  });
  console.log(gasto);
  await gasto.save();
  res.json('status: Gasto guardado');
}

module.exports=gastosControllers;
```

	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

15. Luego se define el esquema para la base de datos. En la carpeta models se crea un script *gastos.js* con los elementos de la colección Gastos

```
const mongoose=require('mongoose');
const {Schema}=mongoose;

const GastosSchema=new Schema({

  tipo:{type:String, required:true},
  monto:{type:Number, required:true},
  descripcion:{type:String, required:true}
})

module.exports=mongoose.model('Gasto',GastosSchema);
```

16. Para devolver todos los gastos que tenemos guardados en la base de datos se usará en el script *gastos.controller.js* el método *gastosController.getGastos*. En el navegador podemos probar directamente la URL <http://localhost:3000/api/gastos> o usando POSTMAN colocar la URL con el método GET.

```
gastosController.getGastos= async(req, res)=>
{
  const gastos= await Gasto.find();
  res.json(gastos);
}
```

17. Para ingresar datos en la base de datos se usará el método *gastosControllers.addGasto*.

```
gastosController.createGastos= async(req,res)=>{
  const gasto=new Gasto(req.body);
  console.log(gasto);
  await gasto.save();
  res.json('status: Gasto guardado');
}
```

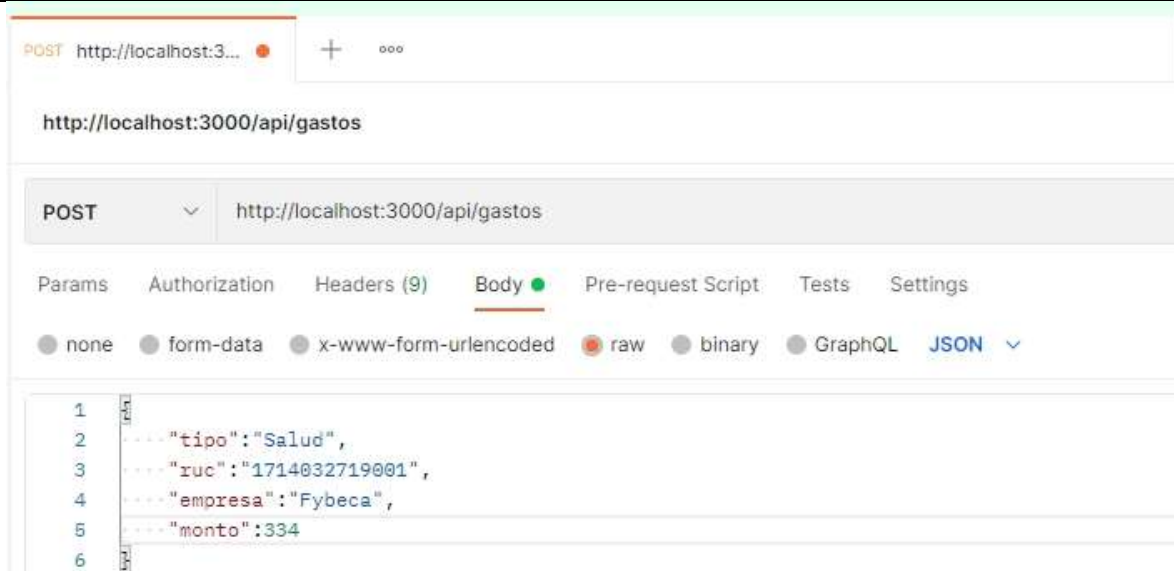
18. Para simular el ingreso de datos mediante un formulario usamos POSTMAN con la URL <http://localhost:3000/api/gastos> y método POST. Colocar en HEADERS el KEY correspondiente según la figura adjunta.

Params Authorization **Headers (9)** Body ● Pre-request Script

Headers 8 hidden

	KEY	VALUE
☒	Content-Type	application/json

19. En la pestaña BODY colocar el archivo JSON correspondiente al modelo definido en la aplicación



20. Para buscar un gasto por el ID correspondiente se usará el método `gastosController.getGasto`

```

gastosController.getGasto=async(req,res)=>{
  console.log(req.params.id);
  const gasto= await Gasto.findById(req.params.id);
  res.json(gasto);
}

```

En el navegador se incluye en el URL el código respectivo <http://localhost:3000/api/gastos/61ddb9642ea140b708f874d>



21. Para realizar la actualización se usará el método `gastos.Controller.editGasto`

```

gastosController.editGasto=async(req,res)=>{
  const {id}=req.params;
  const gasto={
    tipo: req.body.tipo,
    ruc: req.body.ruc,
    empresa: req.body.empresa,
    monto: req.body.monto
  };
  await Gasto.findByIdAndUpdate(id, {$set:gasto},{new: true});
  res.json('status: Gasto actualizado');
}

```

	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

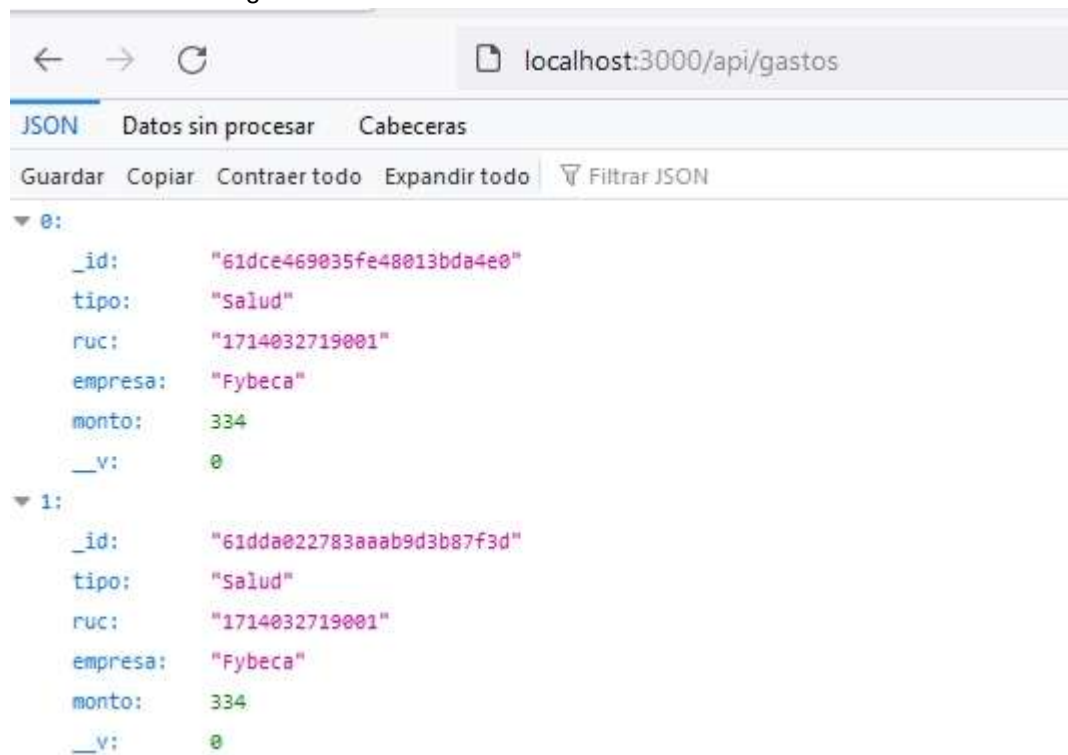
Usando POSTMAN se usa el método PUT se coloca el URL con el ID respectivo <http://localhost:3000/api/gastos/61ddbe9642ea140b708f874d> y en el archivo JSON se realiza la actualización respectiva



22. Crear un método que me permita consultar todos los gastos por tipo: salud, alimentación y educación
23. Incluir el método borrar gasto por ID

RESULTADO(S) OBTENIDO(S):

Consultar todos los gastos



Consultar por id



localhost:3000/api/gastos/61dce469035fe48013bda4e0

JSON

Datos sin procesar

Cabeceras

Guardar Copiar Contraer todo Expandir todo Filtar JSON

```
{
  "_id": "61dce469035fe48013bda4e0",
  "tipo": "Salud",
  "ruc": "1714032719001",
  "empresa": "Fybeka",
  "monto": 334,
  "__v": 0
}
```

Actualizar por id

http://localhost:3000/api/gastos/61ddb6c642ea140b708f874f

PUT

http://localhost:3000/api/gastos/61ddb6c642ea140b708f874f

Params

Authorization

Headers (9)

Body

Pre-request Script

Tests

Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
2   ... "tipo": "Educacion",
3   ... "ruc": "1789732654001",
4   ... "empresa": "Universidad Politécnica Salesiana",
5   ... "monto": 3000
6
7
```


	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

← → ↺

localhost:3000/api/gastos

JSON Datos sin procesar Cabeceras

Guardar Copiar Contraer todo Expandir todo Filtrar JSON

```

▼ 0:
  _id: "61ddbe9642ea140b708f874d"
  tipo: "Salud"
  ruc: "1789732719001"
  empresa: "Saludsa"
  monto: 1200
  __v: 0
▼ 1:
  _id: "61ddbec642ea140b708f874f"
  tipo: "Educacion"
  ruc: "1789732654001"
  empresa: "Universidad Politécnica Salesiana"
  monto: 3000
  __v: 0

```

CONCLUSIONES:

- El uso de una arquitectura MVC permite organizar la funcionalidad de una aplicación de tal modo que el front-end y el back-end se encuentren totalmente separados. Dando la posibilidad de realizar cambios sin afectar a ninguno de ellos

RECOMENDACIONES:

- Se puede revisar información adicional sobre Mongoose en <https://www.npmjs.com/package/mongoose>
- Se puede revisar la información oficial sobre el uso de MongoDB en <https://www.mongodb.com/>

Docente / Técnico Docente:

Firma: _____

